

Programación para Física y Astronomía

Departamento de Física.

Coordinadora: C Loyola

Profesores C Femenías / D Basantes

Primer Semestre 2025

Universidad Andrés Bello

Departamento de Física y Astronomía



Introducción y Repaso

Ajuste lineal con `np.polyfit`

Ejemplo: Ley de Hubble

Cierre

Introducción y Repaso

- **Tema central:** Ajuste de datos y modelado con **NumPy** y **SciPy**.
- **Objetivo:** Construir, paso a paso, una base sólida para ajustar datos experimentales y astronómicos.
- **Hoy (P1):**
 - Ajuste lineal con **np.polyfit**.
 - Ejemplos: Ley de Hubble, relación masa–luminosidad (transformación log–log).
 - Residuos y bondad de ajuste (χ^2 reducido).
- **Prerequisitos suaves:** **numpy** (arrays, operaciones vectorizadas) y **matplotlib**.

Introducción y Repaso $\ni$ Mini-repaso: arrays, ruido y gráficos

```
1 import numpy as np
2 import matplotlib.pyplot as plt
3
4 # Semilla reproducible
5 rng = np.random.default_rng(42)
6
7 # Datos sintéticos simples
8 x = np.linspace(0, 10, 25)
9 y_true = 2.5 * x + 5.0
10 sigma = 2.0 # incertidumbre típica de medición
11 y_obs = y_true + rng.normal(0, sigma, size=x.size)
12
13 plt.scatter(x, y_obs, s=20)
14 plt.xlabel('x'); plt.ylabel('y'); plt.grid(alpha=0.3)
15 plt.title('Datos con ruido')
16 plt.show()
```

Idea: simularemos escenarios físicos con ruido para practicar ajustes.

Ajuste lineal con `np.polyfit`

Ajuste lineal con `np.polyfit` $\ni$ Fenómeno físico: relación lineal simple

- **Escenarios típicos:** calibración sensor $V = aT + b$, ley de Hooke en pequeñas deformaciones $F = kx$, ley de Ohm $V = RI$.
- **¿Por qué una recta?** por *aproximación lineal* (expansión de Taylor de primer orden) en un rango acotado.
- **Qué medimos:** pares (x_i, y_i) con incertidumbre en y y (a veces) en x .
- **Meta:** estimar parámetros físicos (pendiente e intercepto) y evaluar calidad del modelo.

Definición

`np.polyfit(x, y, deg)` ajusta un polinomio de grado **deg** a los datos (x, y) minimizando la suma de los cuadrados de los residuos.

- **Parámetros principales:**
 - **x:** array de valores independientes
 - **y:** array de valores dependientes (observaciones)
 - **deg:** grado del polinomio (1 = lineal, 2 = cuadrático, etc.)
- **Retorna:** coeficientes del polinomio en orden descendente
- Para **deg=1:** retorna $[m, b]$ donde $y = mx + b$

Ajuste lineal con `np.polyfit` $\ni$ Recta por mínimos cuadrados con `np.polyfit`

```
1 import numpy as np
2 import matplotlib.pyplot as plt
3
4 # Supongamos  $y = m x + b + \text{ruido}$ 
5 m_est, b_est = np.polyfit(x, y_obs, deg=1) # deg (int) grado del polinomio a ajustar
6 y_fit = m_est * x + b_est
7
8 plt.scatter(x, y_obs, label='datos')
9 plt.plot(x, y_fit, 'r', label=f'fit: y={m_est:.2f}x+{b_est:.2f}')
10 plt.legend(); plt.grid(alpha=0.3); plt.show()
11
12 print(f'Pendiente: m = {m_est:.3f}')
13 print(f'Intercepto: b = {b_est:.3f}')
```

Notas: `np.polyfit` entrega coeficientes polinomiales por mínimos cuadrados.

Ajuste lineal con `np.polyfit` $\ni$ ¿Qué hace este código? (paso a paso)

- Genera datos x y una relación lineal $y_{\text{true}} = mx + b$ con ruido gaussiano (σ).
- Aplica `np.polyfit` para obtener m, b que minimizan $\sum r_i^2$.
- Grafica datos y la recta ajustada para inspección visual.

Por qué así: es la forma estándar y eficiente de mínimos cuadrados en NumPy.

Ajuste lineal con `np.polyfit` ∋ ¿Qué hace realmente `polyfit`? y supuestos

- **Modelo:** asumimos una relación polinómica (aquí, recta)
 $y_i \approx mx_i + b$.
- **Objetivo:** encontrar m, b que *minimicen la suma de cuadrados* de los residuos: $\sum (y_i - y_{\text{mod}})^2$.
- **Supuestos implícitos:**
 - Errores en y son gaussianos, independientes y con varianza constante.
 - El modelo lineal es adecuado en el rango de datos.
- **Física:** estos supuestos se aproximan cuando la incertidumbre experimental es similar para todos los puntos y el fenómeno es lineal en el rango.
- **Resultado:** `polyfit` devuelve los coeficientes del polinomio que minimiza el error cuadrático medio.

Ajuste lineal con `np.polyfit` $\ni$ Métricas de calidad del ajuste: ¿Qué tan bueno es nuestro modelo?

Pregunta fundamental

Una vez que tenemos un ajuste, ¿cómo sabemos si es bueno?

- **Inspección visual:** útil pero subjetiva
- **Métricas cuantitativas:** nos dan números objetivos

Tres métricas principales

1. **Residuos** (r_i): diferencias punto a punto
2. **Coeficiente** R^2 : fracción de varianza explicada
3. **χ^2 reducido:** consistencia con incertidumbres

Ajuste lineal con `np.polyfit` $\ni$ Métrica 1: Residuos (r_i)

Definición

El residuo es la diferencia entre el valor observado y el predicho por el modelo:

$$r_i = y_{\text{obs},i} - y_{\text{modelo},i}$$

- **Interpretación física:**

- $r_i > 0$: el modelo subestima la observación
- $r_i < 0$: el modelo sobreestima la observación
- $r_i \approx 0$: el modelo predice bien ese punto

- **Análisis de residuos:**

- Patrón aleatorio $\Rightarrow$ modelo adecuado
- Patrón sistemático (curva, tendencia) $\Rightarrow$ modelo inadecuado

- **Ejemplo:** Si ajustamos una recta a datos que siguen una parábola, los residuos mostrarán una forma de "U".

Ajuste lineal con `np.polyfit` $\ni$ Métrica 1:

Cálculo de residuos

```
1 import numpy as np
2 import matplotlib.pyplot as plt
3
4 # Calcular residuos
5 res = y_obs - y_fit
6
7 # Estadísticas básicas de residuos
8 print(f"Media de residuos: {np.mean(res):.6f}") # debe ser = 0
9 print(f"Desviación estándar: {np.std(res):.3f}")
10 print(f"Residuo máximo: {np.max(np.abs(res)):.3f}")
11
12 # Gráfico de residuos vs x
13 plt.figure(figsize=(10, 4))
14 plt.subplot(1, 2, 1)
15 plt.scatter(x, res, alpha=0.6)
16 plt.axhline(0, color='r', linestyle='--')
17 plt.xlabel('x'); plt.ylabel('Residuos'); plt.grid(alpha=0.3)
18 plt.title('Residuos vs x')
19
20 # Histograma de residuos
21 plt.subplot(1, 2, 2)
22 plt.hist(res, bins=10, alpha=0.7, edgecolor='black')
23 plt.xlabel('Residuo'); plt.ylabel('Frecuencia')
24 plt.title('Distribución de residuos'); plt.show()
```

Ajuste lineal con `np.polyfit` $\ni$ Métrica 2: Coeficiente de determinación (R^2)

Definición

R^2 mide la proporción de la varianza en y que es explicada por el modelo:

$$R^2 = 1 - \frac{SS_{\text{res}}}{SS_{\text{tot}}} = 1 - \frac{\sum (y_i - y_{\text{fit},i})^2}{\sum (y_i - \bar{y})^2}$$

- **Componentes:**
 - SS_{res} : suma de cuadrados de residuos (error del modelo)
 - SS_{tot} : varianza total de los datos
 - $\bar{y}$: promedio de las observaciones
- **Rango de valores:** $0 \leq R^2 \leq 1$
 - $R^2 = 1$: el modelo explica perfectamente los datos
 - $R^2 = 0$: el modelo no explica nada (equivalente a usar $\bar{y}$)
 - $R^2 < 0$: el modelo es peor que usar el promedio (raro)

Ajuste lineal con `np.polyfit` $\ni$ Métrica 2:

Cálculo de R^2

```
1 # Calcular componentes
2 res = y_obs - y_fit
3 SS_res = np.sum(res**2) # Suma de cuadrados residual
4 SS_tot = np.sum((y_obs - np.mean(y_obs))**2) # Varianza total
5
6 # Calcular R^2
7 R2 = 1 - SS_res / SS_tot
8
9 print(f"SS_res = {SS_res:.2f}")
10 print(f"SS_tot = {SS_tot:.2f}")
11 print(f"R^2 = {R2:.4f}")
12
13 # Interpretación
14 if R2 > 0.9:
15     print("Excelente ajuste: >90% de varianza explicada")
16 elif R2 > 0.7:
17     print("Buen ajuste: >70% de varianza explicada")
18 elif R2 > 0.5:
19     print("Ajuste moderado")
20 else:
21     print("Ajuste pobre: considerar otro modelo")
```


Ajuste lineal con `np.polyfit` $\ni$ Métrica 2: Interpretación de R^2

- **Ventajas:**

- Fácil de interpretar (porcentaje de varianza explicada)
- Sin unidades (adimensional)
- Útil para comparar modelos en los mismos datos

- **Limitaciones importantes:**

- NO indica si el modelo es apropiado físicamente
- Siempre aumenta al agregar parámetros (sobreajuste)
- NO usa las incertidumbres σ_i de las mediciones

Advertencia física

Un R^2 alto NO garantiza que el modelo sea correcto. Ejemplo: ajustar una recta a datos que siguen una parábola puede dar $R^2 > 0.9$ en un rango limitado, pero el modelo sigue siendo incorrecto.

Ajuste lineal con `np.polyfit` $\ni$ Métrica 3: χ^2 reducido

Definición

El χ^2 reducido compara los residuos con las incertidumbres esperadas:

$$\chi_{\text{red}}^2 = \frac{1}{\nu} \sum_{i=1}^N \frac{(y_i - y_{\text{fit},i})^2}{\sigma_i^2}, \quad \nu = N - p$$

- **Componentes:**

- N : número de puntos de datos
- p : número de parámetros ajustados (2 para recta: m, b)
- ν : grados de libertad = $N - p$
- σ_i : incertidumbre del punto i

- **Diferencia clave con R^2 :**

- χ_{red}^2 Sí usa las incertidumbres σ_i
- Responde: "¿son los residuos consistentes con σ_i ?"

Ajuste lineal con `np.polyfit` $\ni$ Métrica 3:

Interpretación de χ^2 reducido

- Valores típicos e interpretación:

- $\chi_{\text{red}}^2 \approx 1$: **Ideal**. Los residuos son consistentes con las σ_i estimadas.
- $\chi_{\text{red}}^2 \gg 1$: **Problema**. Los residuos son mayores que σ_i . Posibles causas:
 - Modelo inadecuado (falta física)
 - Incertidumbres σ_i subestimadas
 - Datos atípicos (outliers)
- $\chi_{\text{red}}^2 \ll 1$: **Sospechoso**. Los residuos son menores que σ_i . Posibles causas:
 - Incertidumbres σ_i sobrestimadas
 - Sobreajuste (demasiados parámetros)

Regla práctica

Para un buen ajuste: $0.5 < \chi_{\text{red}}^2 < 2.0$

Ajuste lineal con `np.polyfit` $\ni$ Métrica 3:

Cálculo de χ^2 reducido

```
1  # Supongamos que todos los puntos tienen sigma = 2.0
2  sigma = 2.0
3
4  # Calcular chi^2
5  res = y_obs - y_fit
6  chi2 = np.sum((res / sigma)**2)
7
8  # Grados de libertad
9  N = len(y_obs)
10 p = 2  # parámetros: m y b
11 nu = N - p
12
13 # Chi^2 reducido
14 chi2_red = chi2 / nu
15
16 print(f"N = {N} puntos")
17 print(f"p = {p} parámetros")
18 print(f"v = {nu} grados de libertad")
19 print(f" $\chi^2$  = {chi2:.2f}")
20 print(f" $\chi^2_{red}$  = {chi2_red:.2f}")
21
22 if 0.5 < chi2_red < 2.0:
23     print("🟢 Ajuste consistente con incertidumbres")
24 elif chi2_red > 2.0:
25     print("🔴 Residuos mayores que esperado")
26 else:
27     print("🟢 Residuos menores que esperado")
```

Ajuste lineal con `np.polyfit` $\ni$ Comparación de las tres métricas

Aspecto	Residuos	R^2	χ^2_{red}
Usa σ_i	No	No	Sí
Detecta patrones	Sí	No	No
Valor objetivo	≈ 0	≈ 1	≈ 1
Comparar modelos	Difícil	Sí	Sí
Interpretación	Visual	% varianza	Consistencia

Recomendación

- Use **residuos** para detectar problemas del modelo
- Use R^2 para comunicación general
- Use χ^2_{red} para validación estadística rigurosa

Ajuste lineal con `np.polyfit` $\ni$ Ponderación: ¿Por qué pesar los datos?

- **Problema:** No todas las mediciones son igual de confiables.
- **Ejemplo físico:**
 - Medición 1: $y_1 = 10.0 \pm 0.1$ (muy precisa)
 - Medición 2: $y_2 = 10.5 \pm 2.0$ (menos precisa)
- **Pregunta:** ¿Deberían ambas mediciones influir igual en el ajuste?
- **Respuesta intuitiva:** NO. La medición más precisa debe "pesar más" en el resultado final.

Principio físico

Dar más importancia a las mediciones con menor incertidumbre es fundamental para obtener el mejor estimador de los parámetros.

Fórmula de ponderación

El peso de cada punto es inversamente proporcional a su incertidumbre:

$$w_i = \frac{1}{\sigma_i}$$

- Interpretación:

- σ_i pequeño $\Rightarrow w_i$ grande $\Rightarrow$ más influencia
- σ_i grande $\Rightarrow w_i$ pequeño $\Rightarrow$ menos influencia

- Minimización ponderada:

$$\chi^2 = \sum_{i=1}^N w_i^2 (y_i - (mx_i + b))^2 = \sum_{i=1}^N \frac{(y_i - (mx_i + b))^2}{\sigma_i^2}$$

- Los puntos con σ pequeño contribuyen más al χ^2 total.

Ajuste lineal con `np.polyfit` $\ni$ Comparación: Sin ponderación vs Con ponderación (Paso 1)

Paso 1: Generar datos con incertidumbres variables

```
1 import numpy as np
2 import matplotlib.pyplot as plt
3
4 # Datos originales (del ejemplo anterior)
5 rng = np.random.default_rng(42)
6 x = np.linspace(0, 10, 25)
7 y_true = 2.5 * x + 5.0
8
9 # Crear incertidumbres variables para cada punto
10 rng2 = np.random.default_rng(0)
11 sigma_i = rng2.uniform(1.0, 3.0, size=x.size) #  $\sigma$  entre 1 y 3
12
13 # Generar observaciones con ruido proporcional a  $\sigma_i$ 
14 y_obs = y_true + rng.normal(0, 1, size=x.size) * sigma_i
15
16 print("Primeros 5 puntos:")
17 for i in range(5):
18     print(f"x={x[i]:.1f}, y={y_obs[i]:.2f},  $\sigma$ ={sigma_i[i]:.2f}")
```

Resultado: Cada punto tiene diferente nivel de incertidumbre.

Ajuste lineal con `np.polyfit` $\ni$ Comparación: Sin ponderación vs Con ponderación (Paso 2)

Paso 2: Ajuste SIN ponderación

```
1 # Ajuste estándar: todos los puntos pesan igual
2 m_sin, b_sin = np.polyfit(x, y_obs, deg=1)
3 y_fit_sin = m_sin * x + b_sin
4
5 print(f"\nAjuste SIN ponderación:")
6 print(f"  Pendiente m = {m_sin:.3f} (real: 2.500)")
7 print(f"  Intercepto b = {b_sin:.3f} (real: 5.000)")
8
9 # Calcular  $\chi^2$  sin ponderar (asumiendo  $\sigma=1$  para todos)
10 res_sin = y_obs - y_fit_sin
11 chi2_sin = np.sum(res_sin**2)
12 print(f"  Suma de residuos2 = {chi2_sin:.2f}")
```

Problema: Puntos con alta incertidumbre afectan igual que los precisos.

Ajuste lineal con `np.polyfit` $\ni$ Comparación: Sin ponderación vs Con ponderación (Paso 3)

Paso 3: Ajuste CON ponderación

```
1 # Calcular pesos:  $w = 1/\sigma$ 
2 w = 1.0 / sigma_i
3
4 # Ajuste ponderado: puntos precisos pesan más
5 m_con, b_con = np.polyfit(x, y_obs, deg=1, w=w)
6 y_fit_con = m_con * x + b_con
7
8 print(f"\nAjuste CON ponderación:")
9 print(f"  Pendiente m = {m_con:.3f} (real: 2.500)")
10 print(f"  Intercepto b = {b_con:.3f} (real: 5.000)")
11
12 # Calcular  $\chi^2$  ponderado correctamente
13 res_con = y_obs - y_fit_con
14 chi2_con = np.sum((res_con / sigma_i)**2)
15 print(f"   $\chi^2$  ponderado = {chi2_con:.2f}")
```

Resultado: Los parámetros se acercan más al valor real.

Ajuste lineal con `np.polyfit` $\ni$ Comparación: Sin ponderación vs Con ponderación (Paso 4)

Paso 4: Visualización comparativa

```
1 plt.figure(figsize=(10, 6))
2 # Graficar datos con barras de error
3 plt.errorbar(x, y_obs, yerr=sigma_i,
4             fmt='o', alpha=0.6,
5             label='Datos', capsizes=3)
6 # Línea verdadera
7 plt.plot(x, y_true, 'g--', lw=2,
8          label='Modelo real')
9 # Ajuste sin ponderación
10 plt.plot(x, y_fit_sin, 'r-', lw=2,
11          label=f'Sin pond: '
12              f'm={m_sin:.2f}, '
13              f'b={b_sin:.2f}')
```

```
15 # Ajuste con ponderación
16 plt.plot(x, y_fit_con, 'b-', lw=2,
17          label=f'Con pond: '
18              f'm={m_con:.2f}, '
19              f'b={b_con:.2f}')
20 plt.xlabel('x')
21 plt.ylabel('y')
22 plt.legend()
23 plt.grid(alpha=0.3)
24 plt.title('Comparación de ajustes')
25 plt.show()
```

Ajuste lineal con `np.polyfit` $\ni$ Interpretación física de la ponderación

- **Efecto en parámetros:**
 - El ajuste ponderado se "acerca" más a los puntos con σ pequeño
 - Se "aleja" de puntos con σ grande (menos confiables)
- **Cuándo usar ponderación:**
 - Mediciones con diferentes instrumentos o métodos
 - Datos tomados en diferentes condiciones
 - Combinación de observaciones históricas y modernas
- **Advertencia importante:**
 - Si las σ_i están mal estimadas, el ajuste será incorrecto
 - Es crucial conocer bien las incertidumbres experimentales

Regla práctica

Si todos los puntos tienen incertidumbre similar, la ponderación no cambia mucho el resultado.

Ejemplo: Ley de Hubble

Ejemplo: Ley de Hubble $\ni$ Ley de Hubble: contexto físico y observacional

- **Idea simple:** cuanto más **lejos** está una galaxia (d), más **rápido** se aleja (v). La relación es $v = H_0 d$.
- **Interpretación geométrica:** el *pendiente* de la recta es H_0 . Si la expansión fuera perfecta, el *intercepto* sería 0.
- **Régimen de validez local:** para $z \ll 1$, $v \approx cz$. Para z grandes se usa un modelo cosmológico completo.
- **Cómo se mide:**
 - v : a partir del **corrimiento al rojo** z medido en espectros.
 - d : con la **escalera de distancias** (cefeidas, supernovas Ia, etc.).
- **Dispersión real:** las **velocidades peculiares** ($\sim 100\text{--}300$ km/s) añaden ruido, especialmente a distancias pequeñas.

Diccionario rápido (símbolos y unidades)

- v : velocidad de recesión, en km/s.
- d : distancia, en Mpc ($1 \text{ Mpc} \approx 3.26$ millones de años luz).
- H_0 : constante de Hubble, en $\text{km s}^{-1} \text{Mpc}^{-1}$ (~ 70 en este ejemplo).
- z : corrimiento al rojo; si $z \ll 1$, $v \approx cz$ (c : velocidad de la luz).
- Intercepto: valor de v cuando $d = 0$; idealmente ≈ 0 , pero puede verse afectado por sesgos/peculiares.

Ejemplo numérico

Si $H_0 = 70 \text{ km s}^{-1} \text{Mpc}^{-1}$ y $d = 100 \text{ Mpc}$, entonces $v \approx 7000 \text{ km/s}$.

Ejemplo: Ley de Hubble $\ni$ Supuestos, validez y fuentes de dispersión

- **Supuestos:** el universo es *más o menos igual* en todas partes (homogéneo e isótropo) y se expande de forma uniforme *en promedio*.
- **Unidades y tiempos:** H_0 también puede escribirse en s^{-1} . El *tiempo de Hubble* $t_H \approx 1/H_0$ da una escala de edad del universo.
- En el ejemplo siguiente usaremos datos **sintéticos** para practicar el ajuste y recuperar H_0 .

Traducción simple

"Más lejos $\Rightarrow$ más rápido". El H_0 es cuántos km/s aumentan por cada Mpc de distancia.

Cuidado

A distancias pequeñas los términos peculiares pueden dominar y hacer que el intercepto aparente no sea 0.

Ejemplo: Ley de Hubble $\ni$ Ley de Hubble: $v = H_0 d$

```
1 import numpy as np
2 import matplotlib.pyplot as plt
3 # Simular galaxias: velocidad vs distancia
4 rng = np.random.default_rng(7)
5 H0_true = 70.0 # km/s/Mpc
6 d = np.linspace(0, 200, 30) # Mpc
7 v_true = H0_true * d
8 sigma_v = 150.0 # km/s (dispersión peculiar)
9 v_obs = v_true + rng.normal(0, sigma_v, size=d.size)
10
11 # Graficar datos observados
12 plt.scatter(d, v_obs, s=20)
13 plt.xlabel('Distancia (Mpc)'); plt.ylabel('Velocidad (km/s)')
14
15 # Ajustar H0 a los datos simulados
16 H0_hat, v0_hat = np.polyfit(d, v_obs, 1)
17 v_fit = H0_hat * d + v0_hat
18 plt.plot(d, v_fit, 'r', label=f'fit: H0={H0_hat:.1f} km/s/Mpc')
19
20 print(f'H0 estimado = {H0_hat:.1f} km/s/Mpc, intercepto = {v0_hat:.1f} km/s")
```

Física: a primer orden, la expansión se ve como una relación lineal entre v y d .

Ejemplo: Ley de Hubble $\ni$ ¿Qué hace este código de Hubble?

- Simula distancias d y velocidades v con H_0 verdadero, agregando **dispersión peculiar** (ruido) σ_v .
- Ajusta $v = H_0 d + b$ para estimar H_0 e intercepto.
- Muestra la recta resultante y reporta los parámetros.
- Posteriormente calcula residuos y χ^2_{red} para evaluar consistencia con las σ_v .

Conexión física: el pendiente es una medida de la expansión local del Universo.

Ejemplo: Ley de Hubble $\ni$ Ley de Hubble: residuos y χ^2 reducido

```
1 # Residuos y  $\chi^2$  reducido del ajuste lineal  $v = H_0 d + b$ 
2 res = v_obs - v_fit
3 chi2 = np.sum((res/sigma_v)**2)
4 nu = d.size - 2 # parámetros:  $H_0$  y  $b$ 
5 chi2_red = chi2/nu
6 print(f"chi2_red = {chi2_red:.2f}")
7
8 # (Opcional) Inspección de residuos por distancia
9 # for di, ri in zip(d, res):
10 #     print(f"d={di:6.1f} Mpc, residuo={ri:7.1f} km/s")
```

Lectura física: residuos reflejan velocidades peculiares y errores en distancias; $\chi_{\text{red}}^2 \approx 1$ sugiere incertidumbres realistas.

Ejemplo: Ley de Hubble $\ni$ Residuos en contexto físico

- **Residuos** $r_i = v_{\text{obs},i} - v_{\text{fit},i}$: nos dicen cuánto se desvía cada galaxia de la recta de Hubble.
- **Causas típicas**: velocidades peculiares (caídas al pozo de un cúmulo, interacciones), errores en distancias, selección de muestra.
- **Patrones**: si los residuos muestran tendencia con d , el modelo lineal podría ser insuficiente o las incertidumbres están mal estimadas.
- χ^2 **reducido cercano a 1**: sugiere que las σ usadas reflejan bien la dispersión real.

Cierre

- Vimos ajustes lineales con `np.polyfit`.
- Métricas de calidad: Residuos, R^2 , y χ^2_{red} .
- Ponderación por incertidumbres variables.
- Ejemplo físico: Ley de Hubble con análisis completo.
- Próxima sesión (P2): Práctica intensiva con ejercicios de ajustes lineales y análisis de datos.

- NumPy polyfit
- Matplotlib para visualización

¡Buen trabajo hasta la próxima sesión!

- Practicar con datasets propios (p. ej., espectros, curvas de luz, relaciones escalares).
- Traer dudas para profundizar en P2.